Computer Architecture (Practical Class)
Dynamic Memory Allocation

Luís Nogueira Nuno Pereira Raquel Faria

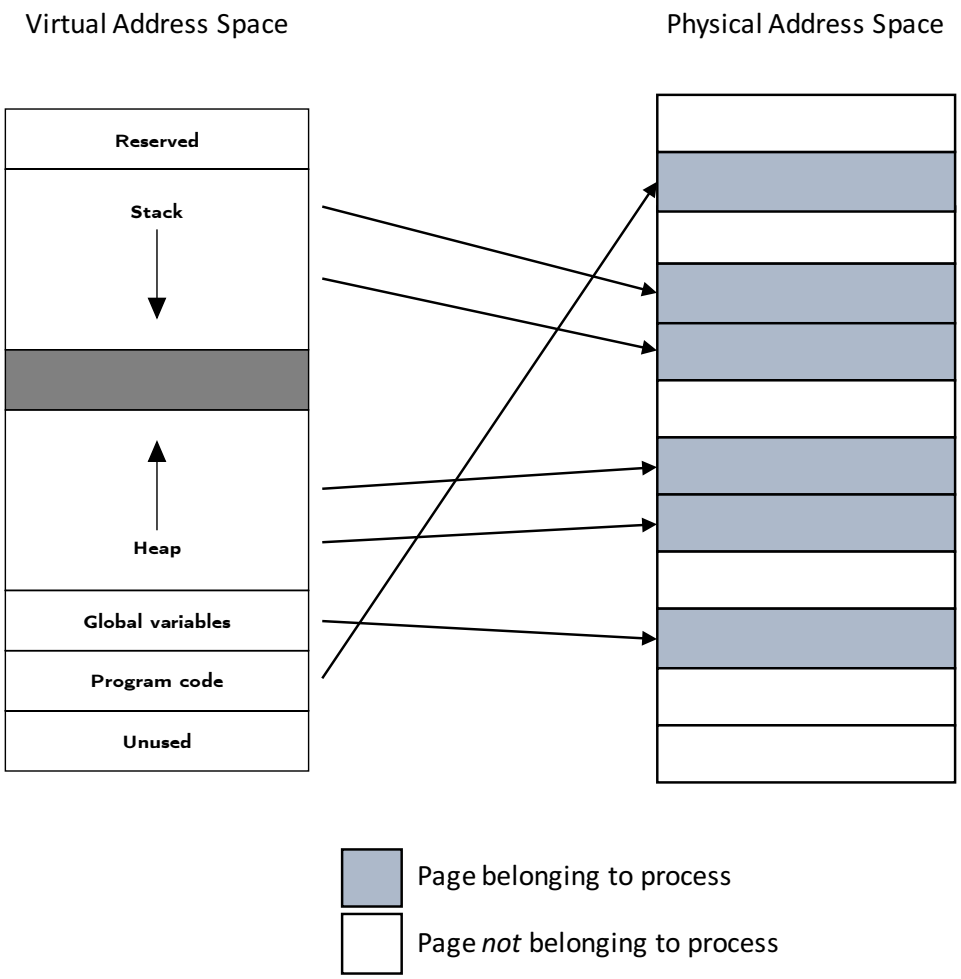
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

{lmn,nap,arf}@isep.ipp.pt

2018/2019

Virtual Memory (1/4)

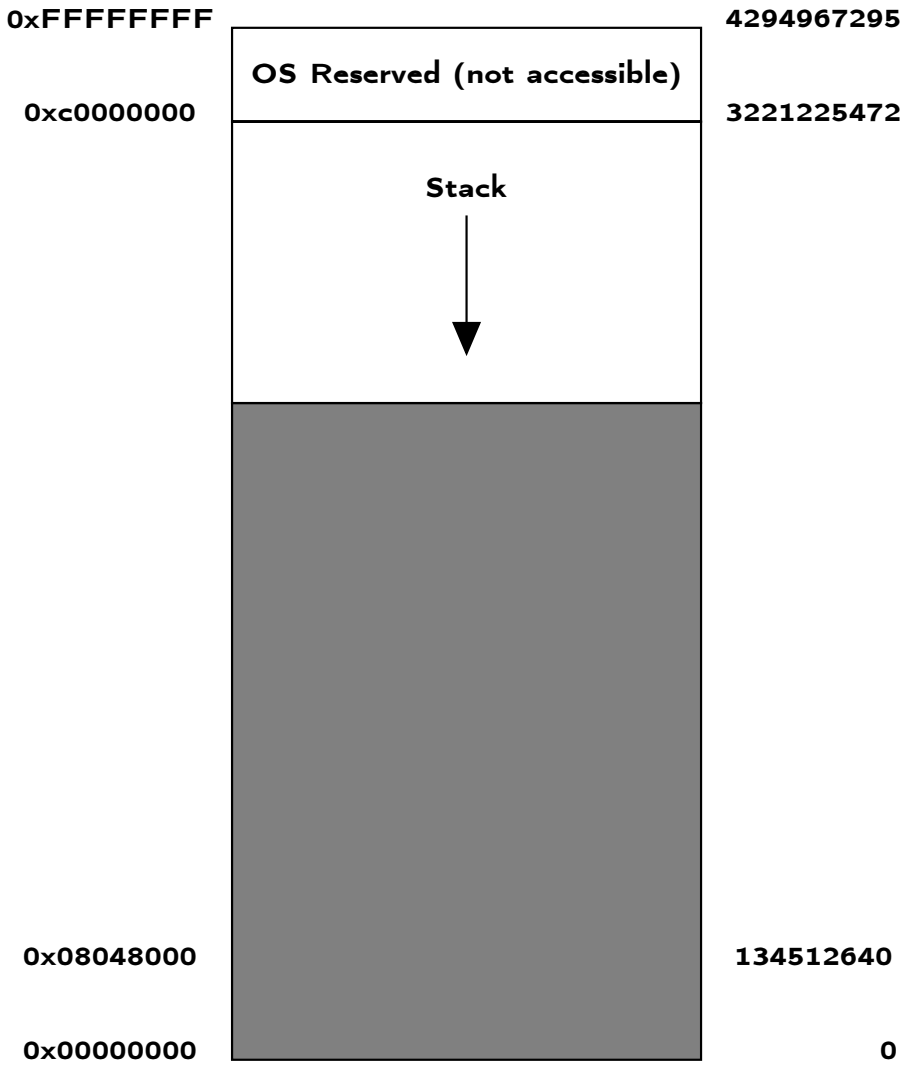
- The operating system gives the illusion that a program[1] has exclusive access to a contiguous memory address space.
- This continuous memory address space is known as the *virtual memory* address space



[1] More precicely, *a process*; you will learn about this distinction in SCOMP.

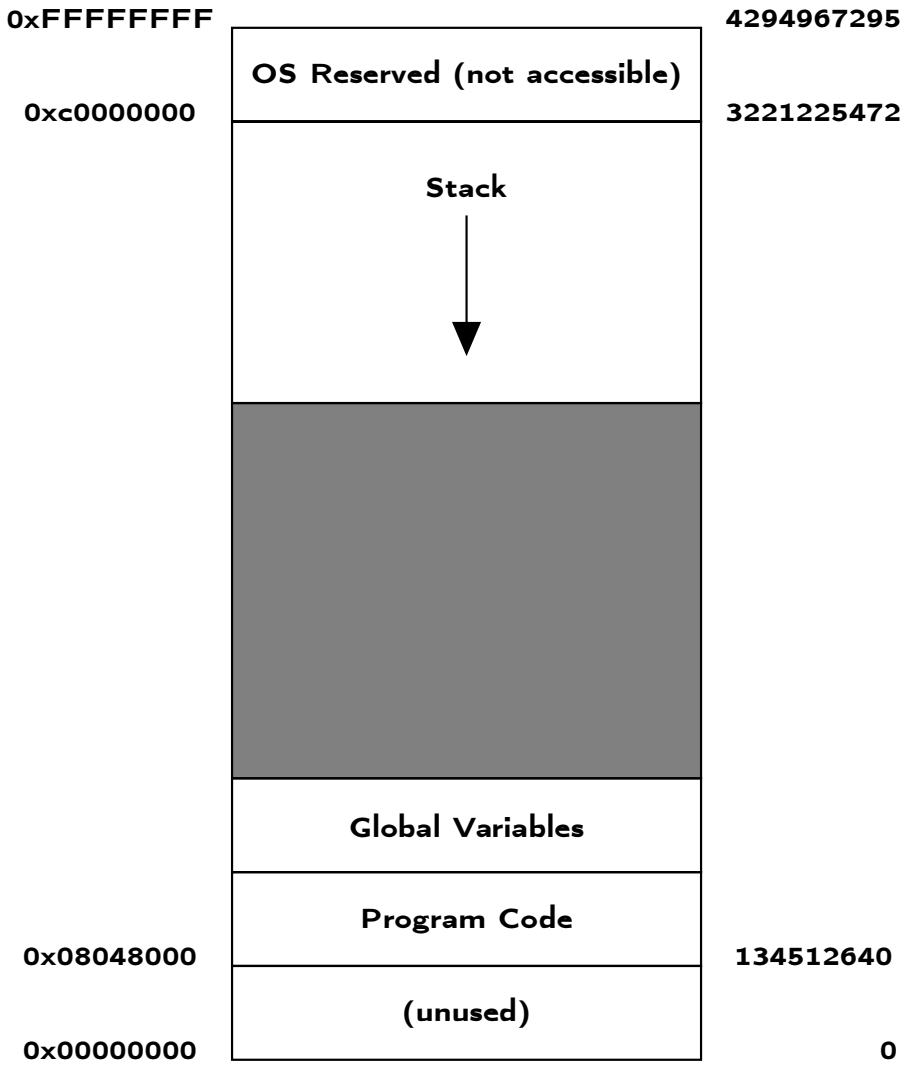
Virtual Memory (2/4)

- The topmost region of the address space is reserved for operating system's code and data.
- At the top of the user's virtual address space is the *stack*.
- The user stack expands and contracts dynamically during the execution of the program. In particular, each time we call a function, the stack grows. Each time we return from a function, it contracts.



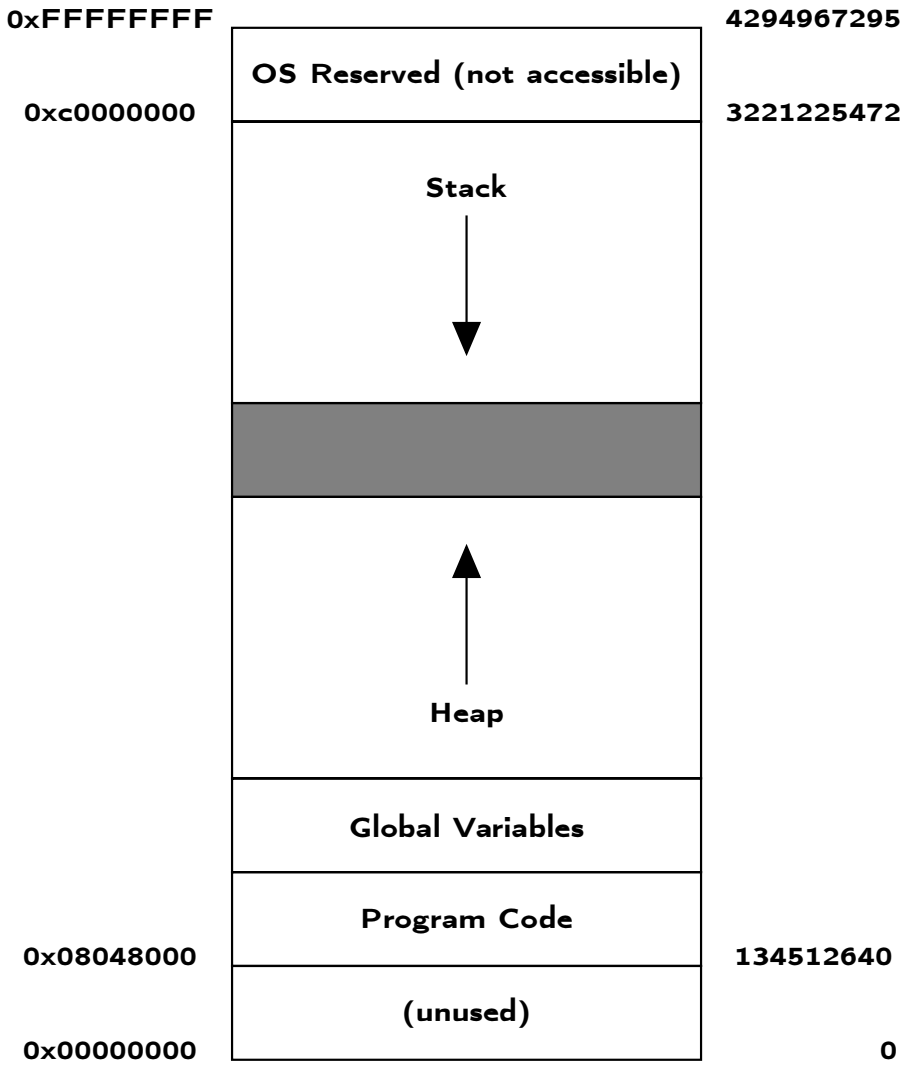
Virtual Memory (3/4)

- The lower region of the address space holds the code and global variables defined by the user's process.



Virtual Memory (4/4)

- The code and global variables are followed immediately by the *heap*.
- The heap expands and contracts dynamically at run time as a result of calls to allocate/free memory



- Allows for a more flexible and efficient memory management, allocating and freeing space on the heap, according to the real needs of the program.

- Motivating example:
  - Imagine a program that stores the ages of students in an array
  - Students are organized in classes, and the maximum number of students per class is 60
  - Assuming three classes (A, B and C), using a static allocation, we would reserve $3 * 60 * 4 = 720$ bytes

Definition of Arrays to Hold Student Ages

```
#define NSTUDENTS 60

int ages_class_A[NSTUDENTS];
int ages_class_B[NSTUDENTS];
int ages_class_C[NSTUDENTS];
```

  - However, the real number of students is 20, 23 and 18 (for A, B and C respectively)
  - The real amount of bytes need is $(20 + 23 + 18) * 4 = 244$ bytes
  - Therefore, $720 - 244 = 476$ bytes are being wasted.

Dynamic Memory Allocation in C (1/4)

The C standard library provides calls to dynamically allocate/free memory on the heap.

- `void* malloc(size_t size);`
  - Allocates a continuous memory block of at least `size` bytes and returns a pointer to it
  - If `malloc()` encounters a problem (e.g., the program requests a block of memory that is larger than the available virtual memory), then it returns NULL
  - The allocated memory is *not initialized*.
- `void free(void *ptr);`
  - Free allocated heap memory blocks previously allocated with a memory allocation call.
  - The `ptr` argument must point to the beginning of an allocated block that was obtained from a memory allocation call.

`malloc()` and `free()` Example

```c
#include <stdlib.h> /* malloc() and free() are part of stdlib */

/* declare pointer */
int *ptr_int=NULL;
/* allocate 10 integers in the heap */
ptr_int=(int *) malloc(10 * sizeof(int));

/* use allocated memory */
ptr_int[0] = 10;
*(ptr_int + 1) = 20;

/* free memory */
free(ptr_int);
```

Dynamic Memory Allocation in C (2/4)

- `void* calloc(size_t n, size_t size);`
  - Reserves a continuous block of memory of at least `n * size` bytes
  - Returns a pointer to the allocated memory block, or `NULL` in error
  - The memory block is initialized to 0

`calloc()` **Example**

```
#include <stdlib.h> /* *alloc(), free() are part of stdlib */

/* declare pointer */
int *ptr_int=NULL;
/* allocate 100 integers in the heap */
ptr_int=(int *) calloc(100, sizeof(int));
...
/* free memory */
free(ptr_int);
```

Dynamic Memory Allocation in C (3/4)

- `void *realloc(void *ptr, size_t size);`
  - Changes the size of a memory block previously allocated with `malloc()` ou `calloc()`
  - Returns a pointer to the allocated memory block, or `NULL` in error
  - Data is kept if the size is increased, or truncated is size is decreased.

`realloc()` **Example**

```
#include <stdlib.h> /* *alloc(), free() are part of stdlib */

/* declare pointer */
int *ptr_int=NULL;
/* allocate 100 integers in the heap */
ptr_int=(int *) calloc(100, sizeof(int));

/* allocate one more integer in the heap */
ptr_int=(int *) realloc(ptr_int, 101 * sizeof(int));

/* free memory */
free(ptr_int);
```

Dynamic Memory Allocation in C (4/4)

Important note about the usage of realloc()

- If `realloc()` fails, the pointer to the memory block is lost!
- Use a temporary pointer and check the return of `realloc()`

Check Return of `realloc()` Example

```
#include <stdlib.h> /* *alloc(), free() are part of stdlib */

/* declare pointers */
int *ptr_int=NULL, ptr_tmp=NULL;
/* number of ints to allocate */
int n=100;

/* allocate n integers in the heap */
ptr_int=(int *) calloc(n, sizeof(int));

/* allocate one more integer in the heap
   NOTE: return is stored in temporary pointer */
ptr_tmp=(int *) realloc(ptr_int,(n+1) * sizeof(int));

/* check realloc() return */
if(ptr_tmp!=NULL){
        ptr_int=ptr_tmp;
        ptr_tmp=NULL;
}

/* free memory */
free(ptr_int);
```

Practice

- Implement, in C, a program that requests the user for integer values and stores them.
- The program should request values until the user enters a negative integer.
- The program starts by reserving space for 10 integers, and increases this space as needed.
- The value read from the user should be copied to the array (at `index`) by the following function, implemented in assembly:
  `void write_value(int *array, int value, int index)`